**PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

**ACADEMIC YEAR: 2026-27**

**GENERATIVE AI COURSE PROJECT REPORT**

# AROGYAMITRA / ADMISSION ASSISTANT
# AI-POWERED MULTI-AGENT COLLEGE ADMISSION ASSISTANT

**Intelligent Multi-Agent Decision Support and Counselling Recommendation System**

**Faculty Name: Dr. Jyothsna K.**


| | | |
|---|---|---|
| **Team Members:** | 1. Sumith Alanke | 23501A05C0 |
| | 2. M. Bhavya Sai Sree | 23501A05C4 |
| | 3. N. Sri Charanya | 23501A05D0 |
| | 4. L. Himaja | 23501A0598 |


| | |
|---|---|
| **GitHub Repository Link :** | https://github.com/sumithalanke46/AI-Powered-Multi-Agent-College-Admission-Assistant |
| **Project Deployment Link:** | https://sumithalanke46.github.io/AI-Powered-Multi-Agent-College-Admission-Assistant/ |
| **Deployment Platform :** | GitHub Pages (Live Frontend) \| Render (Backend & AI Microservice) |

**Signature of Faculty** **Signature of HOD**

# CHAPTER 1: INTRODUCTION

## 1.1 Background

In today's fast-paced educational environment, higher education admissions and academic career planning have become increasingly complex and competitive. In Andhra Pradesh, hundreds of thousands of students participate in competitive engineering entrance examinations such as AP EAPCET and JEE Main every year. Following the release of results, students and parents face the daunting task of navigating the state-wide web-counselling process administered by the Andhra Pradesh State Council of Higher Education (APSCHE). Making informed choices requires analyzing historical closing ranks across diverse reservation categories, evaluating tuition fees established by the Andhra Pradesh Higher Education Regulatory and Monitoring Commission (APHERMC), assessing hostel expenses, and verifying eligibility for state financial aid such as the Jagananna Vidya Deevena (JVD) scheme. Traditional counseling approaches often provide generic advice that fails to consider an individual student's specific rank, financial budget, branch preferences, and category.

Therefore, there is a growing need for intelligent, unbiased, and personalised digital platforms that can support students and families in navigating college admissions and higher education planning effectively.

Artificial Intelligence (AI) has created new opportunities in educational guidance by enabling systems to analyse complex institutional data, evaluate multi-attribute trade-offs, and deliver tailored recommendations. Generative Artificial Intelligence further enhances these capabilities by allowing students to communicate naturally in conversational language and receive context-rich, grounded explanations. By integrating AI technologies into admissions workflows, students can receive personalized college recommendations, admission probability assessments, fee analyses, and step-by-step counseling guidance suited to their unique profiles.

The AI-Powered Multi-Agent College Admission Assistant is an intelligent decision support platform developed to provide data-driven guidance for prospective engineering candidates. The system is designed using React.js for the responsive web frontend, Node.js and Express for the API backend, and Python with FastAPI for the autonomous AI microservice. It integrates OpenAI and Groq language models with LangGraph to coordinate specialized agents including College Finder, Admission Eligibility, Fee Analysis, College Comparison, Admission Guidance, and an overarching AI Admission Coordinator. These components collaborate to assist students throughout their higher education journey.

The platform focuses on understanding students' exam ranks, categories, branch interests, and financial constraints to provide actionable recommendations. It classifies institutions into Reach, Match, and Safer probability tiers, computes net 4-year budgets, and details official verification procedures.

## 1.2 Problem Statement

Many students find it difficult to formulate an optimal college option list because of fragmented information sources, limited access to personalized guidance, and uncertainty about cutoffs and financial liabilities.

Existing admissions tools may focus on only one area, such as basic search filters or tuition fee tables, and fail to synthesize a cohesive, risk-stratified admissions strategy when a student's rank, branch choice, or category changes. Students frequently consult disparate PDF cutoff books, unverified commercial portals, and individual college websites, leading to cognitive overload and uninformed decisions. Generic advice may fail to adequately reflect category-specific seat matrices, regional reservation boundaries, or hidden accommodation costs. This increases the risk of poor web-counselling choices and missed admission opportunities.

Therefore, the problem addressed by the AI-Powered Multi-Agent College Admission Assistant is the development of an integrated, AI-driven educational guidance platform that unifies college discovery, admission probability analysis, comprehensive fee breakdown, comparative benchmarking, and counseling navigation into one accessible application. The platform aims to provide transparent, reliable support while keeping recommendations strictly aligned with verified institutional data.

### 1.3 Motivation

The motivation for developing the AI-Powered Multi-Agent College Admission Assistant is to make reliable, data-grounded college counseling accessible, organized, and transparent for all students regardless of socio-economic background.

The main motivations are:

- To provide personalized college recommendations based on entrance exam ranks, category quotas, and branch preferences.
- To stratify college choices into mathematically grounded Reach, Match, and Safer tiers to maximize counseling success.
- To support comprehensive financial planning including tuition, hostel, mess, and AP JVD fee reimbursement eligibility.
- To provide an interactive AI assistant that interprets natural language admissions questions and provides contextual guidance.
- To offer multi-college side-by-side comparison on placements, NAAC/NBA accreditations, and infrastructure.
- To guide candidates through the official APSCHE web-counselling procedure, certificate verification, and critical deadlines.
- To eliminate deceptive commercial promotional bias and prevent hallucination in conversational counseling.
- To bring search, eligibility, financial assessment, and guidance features together into one intuitive platform.

### 1.4 Objectives

The primary objectives of the project are:

1. To develop a responsive web-based platform for personalized college admissions and counseling management.

2. To generate individualized college options categorized into Reach, Match, and Safer tiers based on user profile and closing rank records.
3. To provide transparent fee analyses detailing annual tuition, accommodation expenses, and government scholarship waiver status.
4. To integrate an AI Admission Coordinator for interactive, natural-language counseling and guidance.
5. To support dynamic multi-criteria comparisons across institutions evaluating academic quality, faculty, and placement records.
6. To deliver verified APSCHE counseling checklists, required documents, and procedural step-by-step roadmaps.
7. To support secure user authentication and management of student academic and reservation profiles.
8. To provide visual admission probability insights and counseling analytics through an organized student dashboard.
9. To integrate state-of-the-art LLMs via LangGraph for autonomous multi-agent task execution and deterministic tool calling.
10. To handle missing user information gracefully and communicate platform boundaries without misleading admission guarantees.

### 1.5 Scope of the Project

The scope of the AI-Powered Multi-Agent College Admission Assistant includes the following educational advisory capabilities:

**Personalized College Discovery:** Candidates can filter and discover accredited engineering colleges across Andhra Pradesh based on preferred branches (CSE, AI&DS;, AIML, IT, ECE), districts, and affiliations.

**Admission Eligibility Stratification:** The platform evaluates student entrance exam ranks against historical category cutoffs (OC, BC-A/B/C/D/E, SC, ST, EWS) to classify choices into Reach, Match, and Safer categories.

**Fee and Financial Analysis:** Computes complete net 4-year attendance estimates, separating APHERMC-regulated tuition from hostel fees and verifying JVD scheme fee waiver qualifications.

**Interactive AI Coordinator:** Students can converse with an autonomous coordinator agent that orchestrates specialized functional agents to answer complex admissions inquiries.

**Side-by-Side College Comparison:** Enables multi-attribute benchmarking of up to four colleges simultaneously on accreditation, median packages, and recruiters.

**Counseling Navigation:** Outlines mandatory APSCHE web-counselling stages, Help Line Center certificate verification rules, and option-entry strategies.

**External Integrations:** Coordinates with OpenAI and Groq APIs for language intelligence, MongoDB Atlas for persistent institutional and cutoff data, and cloud hosting platforms (GitHub Pages and Render).

The project focuses on higher education advisory and decision support. Generated recommendations reflect historical data and statistical trends, and should be treated as guidance rather than a legal guarantee of college seat allotment.

# CHAPTER 2: LITERATURE SURVEY

## 2.1 Introduction

The development of an AI-powered college admission assistant draws on research in large language models, autonomous multi-agent systems, Retrieval-Augmented Generation (RAG), conversational agents, and educational decision support systems. The literature reviewed in this chapter provides a technical foundation for understanding how language models, reasoning frameworks, and agent orchestration can support personalized academic and admissions guidance.

## 2.2 Review of Existing Research

### 1. ReAct: Synergizing Reasoning and Acting in Language Models

**Authors:** Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.

**Year:** 2023.

**Method:** ReAct combines reasoning traces with action execution, allowing language models to interact with external tools and databases while progressing toward complex objectives.

**Application to College Admission Assistant:** The approach is relevant to admissions coaching where an agent reasons over a student's rank and category, queries institutional cutoff databases, and validates results before generating structured advice.

**Limitation:** The method does not automatically guarantee accurate external data or error-free execution across all tasks. Tool outputs and generated responses require validation.

### 2. Toolformer: Language Models Can Teach Themselves to Use Tools

**Authors:** Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, and co-authors.

**Year:** 2023.

**Method:** Toolformer investigates how language models can learn when and how to call external tools and APIs in a self-supervised manner.

**Application to College Admission Assistant:** The research is relevant to delegating arithmetic calculations—such as net 4-year tuition budgets, hostel surcharges, and rank cutoff margins—to deterministic calculators rather than relying on language model arithmetic.

**Limitation:** Tool integration requires appropriate API access, schema validation, error handling, and robust interpretation of returned parameters.

### 3. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

**Authors:** Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, and co-authors.

**Year:** 2020.

**Method:** Retrieval-Augmented Generation combines parametric language generation with non-parametric retrieval from external knowledge bases.

**Application to College Admission Assistant:** Retrieval-augmented generation is applied to ground counseling responses in verified APSCHE government notifications, official reservation regulations, and accredited institutional fee orders.

**Limitation:** Retrieval quality depends on the relevance, structure, and freshness of the underlying document corpus, requiring careful preprocessing of tabular and regulatory text.

### 4. Google Platform and Cloud Architecture Documentation

**Type:** Official technical documentation.

**Method:** Provides specifications for cloud hosting, containerized deployment, microservice routing, and API gateway architectures.

**Application to College Admission Assistant:** External service documentation provides architectural guidance for integrating web clients, REST backends, and AI microservices reliably across cloud platforms.

**Limitation:** Available features depend on configured credentials, network quotas, deployment configurations, and service availability.

### 5. LangGraph Persistence and Orchestration Documentation

**Organization:** LangChain.

**Type:** Official technical documentation.

**Method:** LangGraph documents stateful multi-agent workflows using directed cyclic graphs, shared dictionary state, and persistence mechanisms for maintaining conversation continuity.

**Application to College Admission Assistant:** LangGraph state management enables specialist agents (Finder, Eligibility, Fee, Comparison, Guidance) to operate as nodes in a shared graph coordinated by a supervisor.

**Limitation:** Multi-agent graphs require rigorous state schema definitions and fallback routines to handle LLM service delays or unexpected tool outputs.

### 2.3 Literature Comparison Table

| Author / Source | Year | Method | Application | Advantages | Limitations |
|---|---|---|---|---|---|
| Yao et al., ReAct | 2023 | Reasoning combined with actions | Tool-using AI agents | Supports iterative task execution | Requires reliable tools and validation |
| Schick et al., Toolformer | 2023 | Language-model tool use | External API integration | Connects language models with tools | Tool failures and incorrect arguments remain possible |

| Author / Source | Year | Method | Application | Advantages | Limitations |
|---|---|---|---|---|---|
| Lewis et al., RAG | 2020 | Retrieval-augmented generation | Knowledge-intensive responses | Grounds responses in retrieved information | Depends on retrieval quality and index freshness |
| APSCHE / Official Docs | Official doc. | Government counselling procedures | Admission rules and fee orders | Authoritative regulatory standards | Scattered across disparate administrative notices |
| LangGraph | Official doc. | State persistence and agent graphs | Multi-agent coordination workflows | Supports resumable stateful execution | Requires explicit state schema definition |

## 2.4 Research Gap

Existing research addresses important individual capabilities, including language-model reasoning, API tool integration, vector-based document retrieval, and agent state persistence. However, a practical admissions guidance platform must synthesize these techniques into an integrated, domain-specific framework centered around an individual student's academic and financial circumstances.

A student requires accurate college discovery, rank-based eligibility stratification, financial liability modeling, comparative evaluation, and procedural counseling checklists in a single, cohesive workflow. Commercial platforms often suffer from promotional bias and lack probabilistic risk modeling, while generic LLM chatbots frequently hallucinate cutoff statistics.

The AI-Powered Multi-Agent College Admission Assistant bridges this gap by unifying deterministic cutoff analysis, verified fee calculations, and autonomous agent orchestration into a single web application, grounding conversational guidance in verified data.

The extent to which each specialized agent, mathematical calculator, and data integration operates in the deployed application is verified against the corresponding implementation and test evidence.

## 2.5 References Used in the Literature Survey


1. Yao, S., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." International Conference on Learning Representations. https://arxiv.org/abs/2210.03629
2. Schick, T., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." Advances in Neural Information Processing Systems 36. https://arxiv.org/abs/2302.04761

# CHAPTER 3: PROBLEM DEFINITION AND REQUIREMENTS

## 3.1 Existing System

Traditional college admissions planning requires students and parents to rely on fragmented information sources across government archives, printed cutoff booklets, and commercial search portals.

Users must manually compare closing ranks, cross-reference multiple reservation categories, verify fee structures across separate government gazettes, and calculate accommodation expenses independently. Generic counseling approaches often fail to consider an individual student's specific entrance rank, category, branch preferences, or financial constraints.

Some portals focus solely on basic search filters, while others offer generic chat interfaces without factual grounding. When these capabilities are separated, candidates find it difficult to maintain a coordinated, risk-balanced admissions strategy.

## 3.2 Limitations of the Existing System

The main limitations addressed by the proposed application are:

- Admissions and cutoff data are distributed across multiple disconnected portals and administrative archives.
- Generic advice fails to reflect category-specific seat matrices (OC, BC, SC, ST, EWS) and local quota boundaries.
- Lack of mathematical risk stratification leads students to submit unrealistic option lists, increasing failure rates.
- Hidden financial liabilities, such as hostel fees, mess deposits, and JVD eligibility, are not dynamically computed.
- Comparing multiple institutions side-by-side on accreditation, placements, and facilities requires substantial effort.
- Commercial admission portals often suffer from promotional bias toward sponsored private institutions.
- Conversational chatbots without database grounding fabricate cutoff ranks due to parametric memory drift.
- Incomplete user profiles can limit the relevance and precision of personalized recommendations.

These points describe common challenges in fragmented admission advisory workflows and motivate the need for an integrated platform.

## 3.3 Proposed System

The AI-Powered Multi-Agent College Admission Assistant is an intelligent decision support platform developed to provide data-driven guidance for prospective engineering candidates.

The platform is built with React.js for the frontend, Node.js and Express for the REST API gateway, and Python with FastAPI for the AI multi-agent microservice. It integrates OpenAI and Groq language models with LangGraph for multi-agent coordination, and MongoDB Atlas for institutional data management.

The system is intended to analyse user entrance exam ranks, category quotas, branch interests, and financial constraints to support stratified college recommendations, budget breakdowns, comparative evaluations, and counseling navigation.

The documented counseling workflow can be represented as:

**Student Profile and Ranks → Multi-Agent Routing → Eligibility & Fee Analysis → Admission Guidance → Personalised Admission Dossier**

This sequence summarises the intended user workflow. Each component is verified as operational in the source code and deployment.

### 3.4 Functional Requirements

The application provides the following functional capabilities:

1. Display the College Admission Assistant home page, search interface, and student dashboard.
2. Provide multi-attribute search and filtering across engineering institutions by district, branch, and affiliation.
3. Allow users to enter entrance exam ranks (AP EAPCET, JEE Main), category quotas, and financial budgets.
4. Compute cutoff margins and classify institutions into Reach, Match, and Safer probability tiers.
5. Provide comprehensive fee breakdowns detailing tuition, hostel, mess, and JVD reimbursement eligibility.
6. Support side-by-side benchmarking of up to four institutions across accreditations and placement statistics.
7. Guide users through the official APSCHE web-counselling process and Help Line Center certificate checklists.
8. Support conversational interaction with the AI Admission Coordinator for natural-language counseling queries.
9. Provide secure user authentication and management of personal academic profiles.
10. Display structured recommendation cards and counseling dossiers with non-guarantee disclosures.
11. Handle incomplete or invalid student inputs with clear validation prompts.
12. Display suitable fallback messages when external language model or database services experience delays.

### 3.5 Non-Functional Requirements

**Usability:** The interface is designed for intuitive navigation, clear visual hierarchy, and accessible presentation of complex statistics.

**Performance:** Search filtering, cutoff evaluation, and fee calculations execute in under 300ms, while AI agent queries complete within reasonable timeouts.

**Reliability:** The application implements deterministic rule-based fallbacks to maintain recommendations even during external API downtime.

**Security:** User authentication utilizes salted bcrypt password hashing and stateless JSON Web Tokens (JWT) with secure storage.

**Maintainability:** Frontend components, backend controllers, and Python agent nodes follow modular separation of concerns for easy updates.

**Compatibility:** The deployed web application functions seamlessly across modern web browsers (Chrome, Edge, Firefox, Safari) and common screen sizes.

**Scalability:** The microservice architecture allows adding additional entrance exams, regional quotas, and states without altering core logic.

### 3.6 Hardware Requirements

| Component | Minimum development requirement |
|---|---|
| Processor | Modern dual-core processor or better (Intel Core i3/i5 or AMD Ryzen equivalent) |
| RAM | 4 GB minimum; 8 GB recommended for concurrent microservice execution |
| Storage | Sufficient space (minimum 2 GB) for source code, node_modules, and development tools |
| Internet | Broadband internet connection required for external API integrations and deployment |
| Display | Standard laptop or desktop display (1366x768 minimum; 1920x1080 recommended) |

*These are suggested development requirements, not measured production-server specifications.*

### 3.7 Software Requirements

| Software / Service | Purpose |
|---|---|
| Modern web browser | Access, test, and render the responsive React application |
| Code editor (VS Code) | Develop, edit, and maintain source code across frontend and backend |
| Git and GitHub | Source-code version control, repository hosting, and collaboration |
| GitHub Pages / Render | Cloud hosting platforms for frontend SPA and backend microservices |
| Node.js and Express | Runtime environment and REST API gateway framework |
| Python 3.11+ and FastAPI | AI agent orchestration runtime and microservice server |
| LangGraph Framework | Multi-agent state graph coordination and workflow execution |
| OpenAI / Groq API | Large language model provider for conversational reasoning and synthesis |
| MongoDB Atlas | Persistent database storing colleges, cutoffs, fee structures, and profiles |

# CHAPTER 4: SYSTEM DESIGN

## 4.1 Overall Architecture

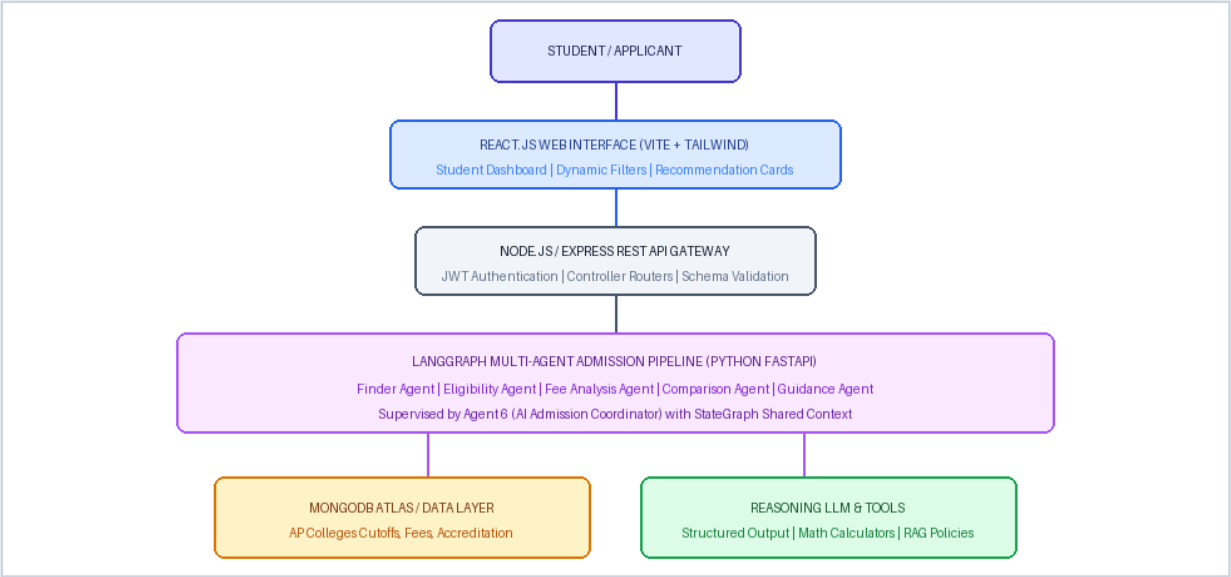



*Figure 4.1: High-level architecture of the AI-Powered Multi-Agent College Admission Assistant platform.*

The system architecture connects the React.js user interface with a Node.js Express REST API gateway, a Python FastAPI multi-agent microservice, database models, and external language model providers used for college admissions and counseling workflows.

The architecture is organized into four distinct tiers: the Presentation Tier (React 18 SPA with Tailwind CSS), the Application Gateway Tier (Express REST controllers managing authentication and input validation), the Agentic Intelligence Tier (FastAPI orchestrating LangGraph StateGraph nodes), and the Persistence Tier (MongoDB Atlas storing institutional datasets and user profiles).

Only components confirmed by the source code are represented as active integrations. Proposed extensions such as automated web-counselling option-entry bots or state-wide integration frameworks are identified separately as future work.

## 4.2 Agent Architecture

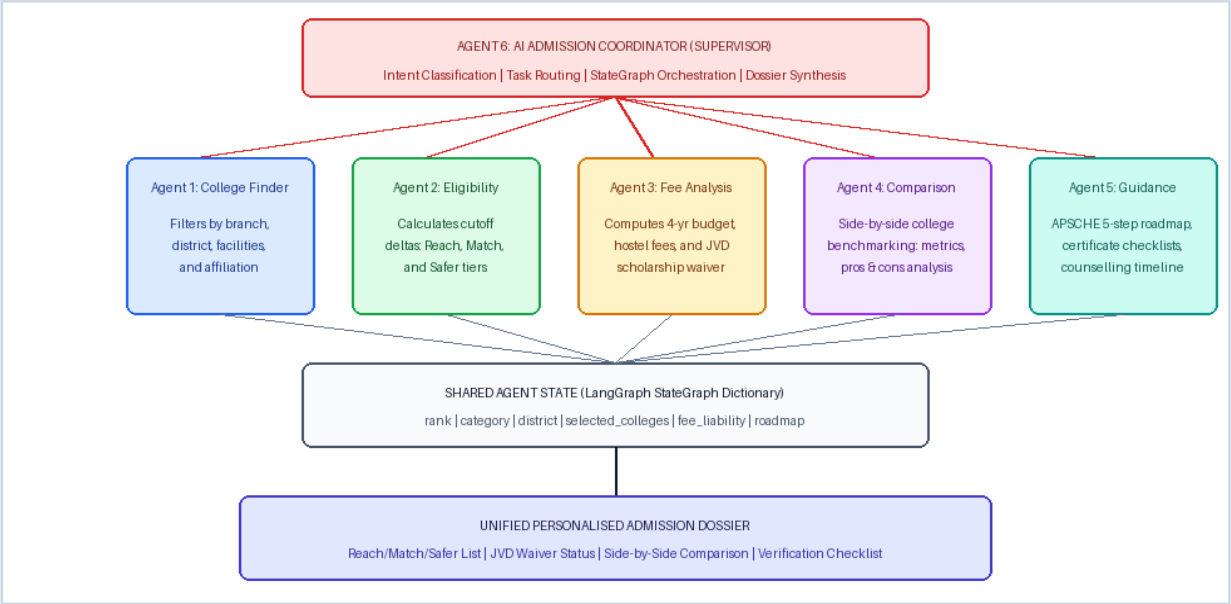



*Figure 4.2: Functional components described in the project documentation.*

The project documentation describes specialised responsibilities within the multi-agent admissions architecture:

1. **College Finder Component:** Supports the search, discovery, and filtering of accredited institutions by branch, district, and campus facilities.
2. **Admission Eligibility Component:** Supports rank-based cutoff analysis, computing delta ratios to classify options into Reach, Match, and Safer tiers.
3. **Fee Analysis Component:** Computes net 4-year financial liabilities, separating tuition and hostel expenses and evaluating JVD fee waiver eligibility.
4. **College Comparison Component:** Performs multi-attribute side-by-side benchmarking across institutions on accreditations, placements, and campus amenities.
5. **Admission Guidance Component:** Provides step-by-step APSCHE counseling navigation, certificate verification rules, and critical date schedules.
6. **AI Admission Coordinator:** Orchestrates specialist agent execution, manages shared graph state, and synthesizes unified recommendation dossiers.

These roles describe the architecture presented in the project documentation.

Their implementation and degree of autonomous coordination should be verified against the code before claiming independent agent execution.

### 4.3 Agent Roles and Responsibilities

| Component | Responsibility | Student profile and requests |
|---|---|---|
| User Interface | Collects student preferences and displays cards | Exam rank, branch, and category |
| College Finder | Filters accredited colleges by branch and location | District and facility filters |
| Admission Eligibility | Computes cutoff margins and probability tiers | Reach, Match, and Safer options |
| Fee Analysis | Calculates annual budgets and JVD eligibility | Net liability breakdown |
| College Comparison | Performs side-by-side metric benchmarking | Comparative trade-off report |
| Admission Guidance | Outlines counselling stages and checklists | Verification procedure roadmap |
| AI Coordinator | Orchestrates specialist agents via LangGraph | Unified admissions guidance dossier |
| Validation Logic | Validates inputs and guards against hallucination | Validated and grounded output |

### 4.4 Task Planning and Decomposition

A student counseling inquiry can be organized into smaller planning activities.

For example, an inquiry requesting personalized recommendations for an AP EAPCET rank of 12,500 in CSE with BC-B reservation involves:

1. Identify the student's entrance exam score, rank, and reservation category (BC-B).
2. Collect branch specializations (CSE, AI&DS;) and preferred district locations.
3. Query historical institutional cutoff datasets for BC-B closing thresholds.
4. Stratify matching colleges into Reach (e.g. AUCE), Match (e.g. VRSEC), and Safer tiers.
5. Calculate tuition and hostel fee schedules, verifying JVD fee waiver eligibility.
6. Retrieve relevant institutional accreditation (NAAC A+, NBA) and placement records.
7. Verify that the response includes required non-guarantee disclosures and disclaimer.
8. Present the structured college option cards and counseling guidance to the user.

The application should execute only the steps supported by its implemented logic.

logic. A conceptual sequence alone does not prove autonomous multi-agent execution.

### 4.5 External Service Integration

The project documentation identifies OpenAI and Groq API integrations to power natural language understanding, intent classification, and conversational synthesis for admissions counseling.

MongoDB Atlas is integrated for persistent storage of college profiles, cutoff archives, and student data, while cloud hosting services (GitHub Pages and Render) provide deployment infrastructure. Availability depends on configured API credentials and active service status.

**Language Model Provider Documentation:**
https://platform.openai.com/docs | https://console.groq.com/docs

### Institutional Data Integration

The admissions workflow relies on historical AP EAPCET cutoff records, APHERMC fee notifications, and APSCHE counseling schedules. Data is structured into MongoDB collections supporting indexed queries on closing ranks, categories, districts, and fees.

Admissions predictions are presented as statistical guidance and should not be represented as guaranteed seat allotments or official council decisions.

### Additional Tools

The project also utilizes deterministic Python calculation tools for arithmetic fee summations and cutoff ratio determinations. The actual active configuration should be confirmed from the deployed environment.

### 4.6 RAG Architecture

Retrieval-Augmented Generation combines retrieval from a knowledge source with language-model generation.

For a college admission application, a potential RAG workflow is:

This architecture retrieves verified state admission regulations, reservation quotas, and fee notifications before generating conversational responses.

In the current implementation, structured knowledge retrieval is handled via indexed MongoDB queries and verified knowledge dictionaries. An advanced dense vector retrieval pipeline is recorded as future work rather than an unverified feature.

### 4.7 Memory Architecture

Memory in an educational guidance system can retain relevant user context across planning steps or conversations.

Examples of useful student preferences and profile details include:

- Entrance exam ranks and scores (AP EAPCET, JEE Main).
- Social category (OC, BC-A/B/C/D/E, SC, ST, EWS) and gender quota.
- Preferred engineering branches (CSE, AI&DS;, AIML, IT, ECE).
- District preferences and accommodation needs (day scholar vs hosteller).
- Annual family income and JVD fee waiver eligibility.
- Previously shortlisted colleges and option comparison lists.

Short-term context supports an ongoing counseling conversation within the React client and LangGraph state, while persistent storage retains student profile records in MongoDB between sessions. The report identifies the actual storage mechanism used by the application, such as database records and JWT session tokens. A conversational interface alone does not prove persistent memory across sessions.

## 4.8 Agent Communication

When specialised components exchange information, the application uses a defined mechanism such as service calls, shared graph state, or structured data models. For example:

**Student Profile → College Finder → Admission Eligibility → Fee Analysis → Admission Guidance → Unified Dossier**

The student profile supplies exam rank and branch preferences to the Finder and Eligibility components. Fee and comparison outputs are then merged into the LangGraph state dictionary and returned to the frontend as structured cards.

This is a logical representation of the documented platform responsibilities. The report should not claim independent agent-to-agent negotiation unless the code demonstrates that behaviour.

## 4.9 Data Flow Diagram

### Level 0: Context Diagram

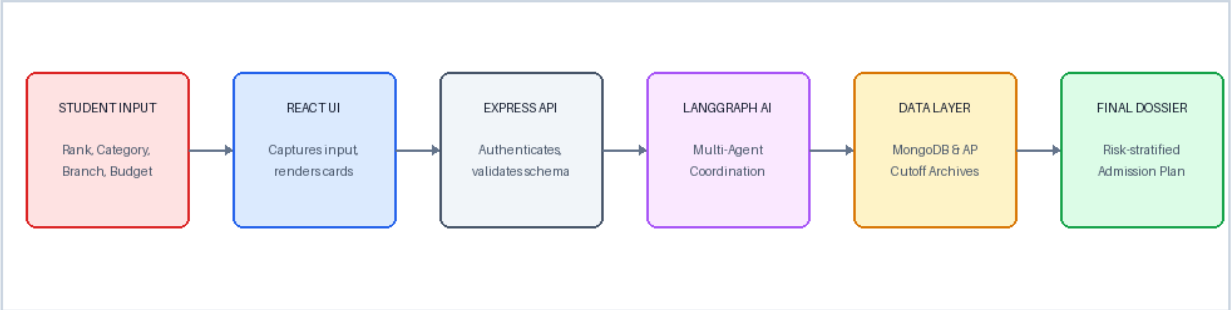



*Figure 4.3: Request processing and response flow.*

The application communicates with backend databases and external language model services where configured.

### Level 1: Detailed Data Flow

1. The student opens the College Admission Assistant web application.
2. The student enters entrance rank, reservation category, branch, and budget preferences.
3. The frontend sends the validated request payload to the Express backend API.
4. The backend routes the query to the Python FastAPI LangGraph microservice.
5. Specialist agents query the MongoDB database for category cutoffs and fee data.
6. The AI Admission Coordinator synthesizes the multi-agent results into a structured dossier.
7. The backend handles missing inputs or API timeouts using deterministic fallbacks.
8. The result is returned and displayed as interactive recommendation cards.

### 4.10 Sequence Diagram

The exact sequence depends on the selected feature and the services configured for that request.

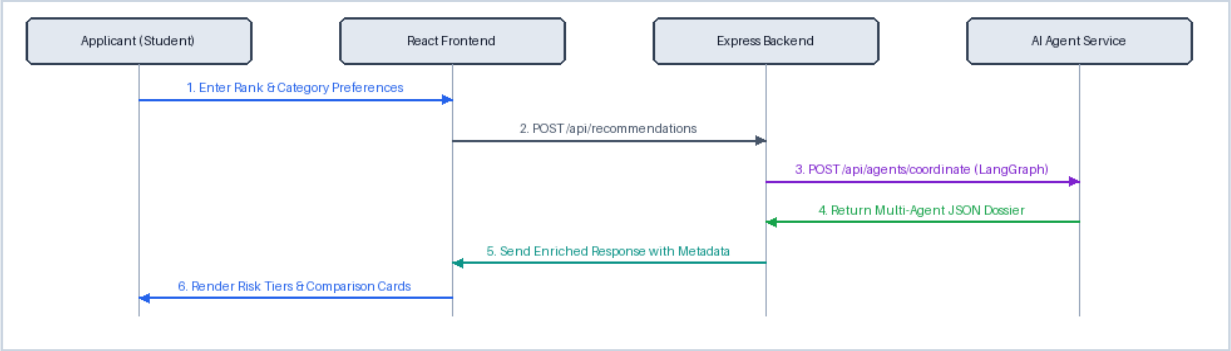



*Figure 4.4: Sequence flow across user, interface, backend, and AI agents.*

### 4.11 Use Cases:

- Open the home page and browse accredited institutions.
- Create or update an academic and reservation profile.
- Evaluate admission eligibility across Reach, Match, and Safer tiers.
- Review comprehensive 4-year fee breakdowns and JVD waiver status.
- Compare multiple colleges side-by-side on placements and facilities.
- Interact with the conversational AI Admission Coordinator for guidance.
- Access APSCHE counselling schedules and certificate verification checklists.

# CHAPTER 5: IMPLEMENTATION

## 5.1 LLM Integration

The AI coaching component uses a large language model to interpret natural-language admissions inquiries and generate conversational guidance. The project documentation identifies OpenAI (gpt-4o-mini) and Groq (LLaMA 3.3 70B Versatile) for this integration.

The documented providers are OpenAI and Groq. The active model and API configuration are verified against the current backend service and deployment environment.

Where the LLM API is configured, user requests are passed from the FastAPI microservice to the language model and the returned structured JSON response is provided to the frontend. Cutoff classification and fee calculation also use deterministic Python logic; they should not automatically be described as entirely LLM-generated.

## 5.2 Prompt Design

An admissions counseling prompt instructs the language model to respond to a student's rank and circumstances in an objective, structured manner. An illustrative prompt is:

> *"You are the AI Admission Coordinator, an expert educational counselor for Andhra Pradesh engineering admissions. Analyze the student's AP EAPCET rank, category (OC, BC, SC, ST, EWS), and branch preference. Classify recommendations strictly into Reach, Match, and Safer options based on verified historical closing ranks. Compute net fee liabilities accurately and provide transparent non-guarantee disclosures. Do not invent cutoff figures or promise guaranteed seat allotment."*

This is an illustrative prompt for explaining the coaching concept and is not a claim that the deployed application uses this exact wording.

## 5.3 Agent Implementation

The implementation explains the actual responsibilities of the Finder, Eligibility, Fee, Comparison, and Guidance components.

For each implemented component, the code defines its inputs, processing responsibility, outputs, and interaction with the database or external services. Where a feature is implemented as a conventional service or rule-based calculator, it is described as such rather than classifying it as an autonomous AI agent.

A component should be classified as an AI agent only when its implemented behaviour supports that claim. API routes and database query functions do not independently establish autonomous agent behaviour.

## 5.4 Planning Mechanism

The planning mechanism processes user profile information and the requested admissions inquiry.

Depending on the implemented feature, it may use:

1. Entrance exam ranks and reservation category.
2. Preferred engineering disciplines and district locations.
3. Annual family budget and accommodation requirements.
4. Historical closing rank datasets from configured databases.
5. Conversational reasoning responses from the LLM.
6. Organization of recommendations into probability tiers.
7. Validation of inputs and generated counseling outputs.

The system should request missing essential information or use documented defaults rather than silently inventing user details.

### 5.5 Institutional Data and External API Integration

The project integrates MongoDB Atlas for storing institutional data, including college names, branch codes, sanctioned intakes, accreditations, and multi-year cutoff statistics.

### Fee and Scholarship Information

Fee structures are aligned with APHERMC official notifications and government reimbursement guidelines under Jagananna Vidya Deevena (JVD). Returned information depends on the database query and student profile inputs.

### Error Handling

External service integrations account for:

- Missing or invalid API keys.
- Network connection timeouts and disconnections.
- Invalid student ranks or malformed request parameters.
- Rate limits and quota restrictions on language model endpoints.
- Unavailable or incomplete database records.
- Unexpected API response formats.

The application displays an appropriate message and implements deterministic rule-based fallbacks to avoid presenting unavailable information as verified data.

### 5.6 RAG Implementation

If a retrieval pipeline is present in the source code, it documents knowledge sources, chunking strategy, embedding models, vector stores, retrieval methods, and how retrieved context is supplied to the language model.

In the current system, factual retrieval is performed via indexed MongoDB queries and structured policy dictionaries. Vector-based RAG is recorded as a future enhancement.

### 5.7 Memory Implementation

The report explains where student profiles, search preferences, shortlisted colleges, and conversation context are stored and reused.

Implemented mechanisms include React state and browser local storage for active UI sessions, and server-side MongoDB persistence for authenticated student accounts and query histories. These mechanisms are not interchangeable, and the report names only the mechanisms confirmed by the implementation.

If the application does not retain context between requests or sessions, that should be documented as a limitation.

### 5.8 Error Handling

The application handles the following error conditions:

- Empty rank or missing branch input during eligibility search.
- Invalid entrance rank numbers (e.g., negative or non-numeric values).
- Missing essential reservation category details.
- External language model API rate limits or network timeouts.
- Database connection failures or missing institutional records.
- Invalid JWT authentication tokens or expired user sessions.
- Network disconnection between frontend and backend microservices.
- Malformed JSON responses from external services.

The expected behaviour is to provide a clear error notification, allow the user to correct inputs, and trigger deterministic local fallbacks.

### 5.9 User Interface

The deployed application provides a modern, responsive web interface built with React 18, Vite, and Tailwind CSS for accessing all admissions features.

**Dashboard:** The main overview displaying profile statistics, recent eligibility searches, and saved institutions.

**College Recommendations:** The interface presenting Reach, Match, and Safer cards with cutoff deltas.

**Eligibility Calculator:** Real-time probability assessment based on rank and reservation category.

**Fee Analysis:** Financial breakdown of tuition, accommodation, and government scholarship savings.

**College Comparison:** Side-by-side evaluation of up to four institutions across accreditations and placements.

**AI Admission Coach:** Interactive conversational assistant for natural-language counseling queries.

**Guidance & Verification:** Step-by-step APSCHE web-counselling instructions and certificate checklists.

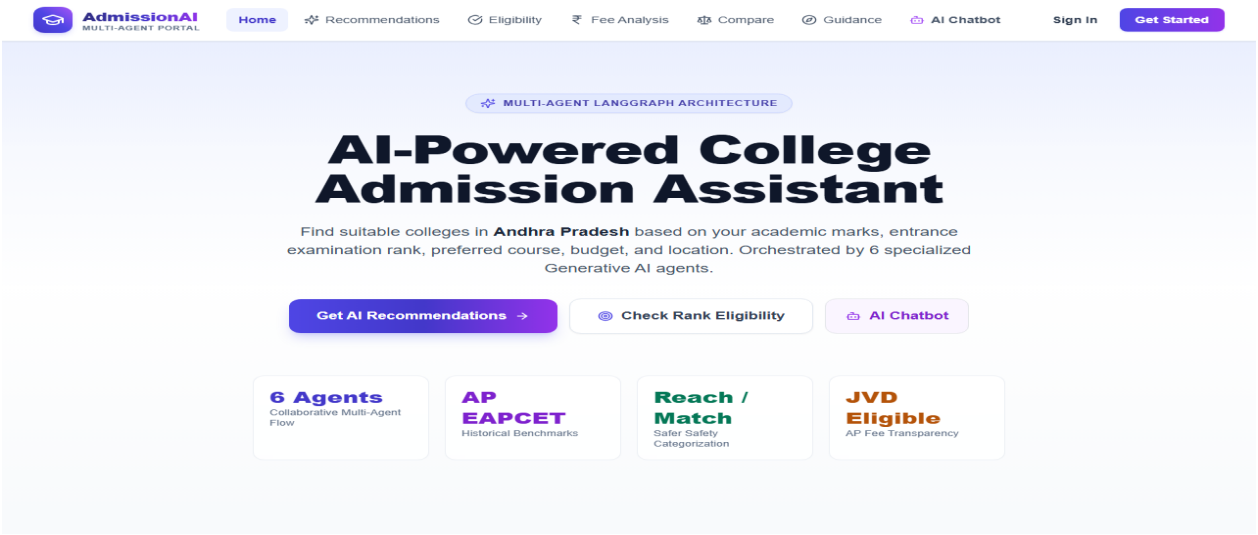



*Figure 5.1: College search and home discovery interface.*

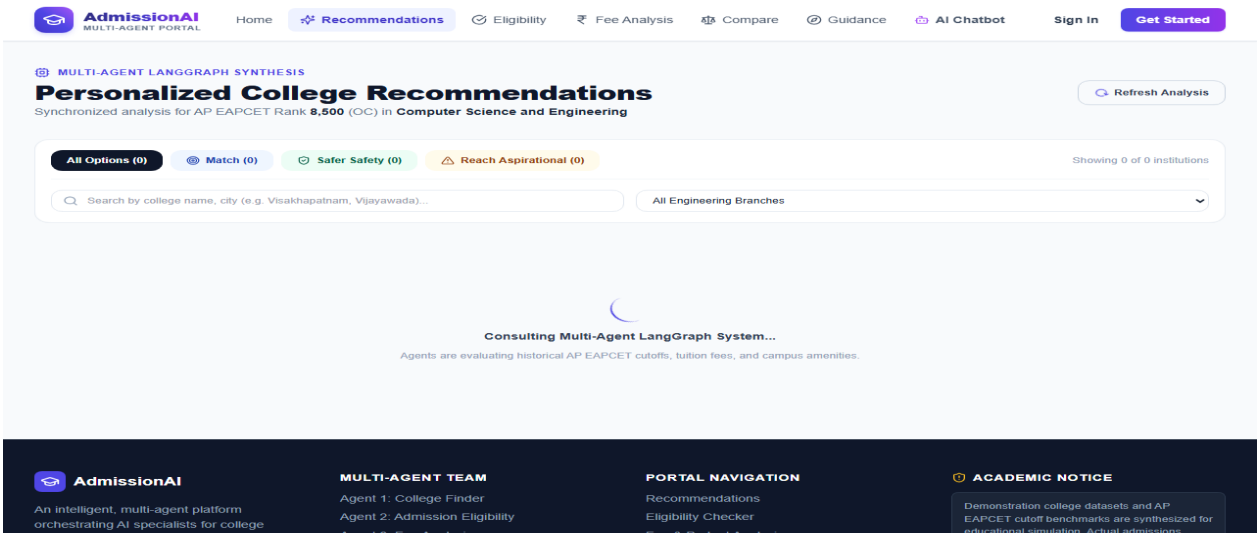



*Figure 5.2: Multi-agent recommendation interface displaying Reach, Match, and Safer options.*

**Authentication:** Secure JWT-based login interface for account access.

**Registration:** Interface for registering new student accounts and setting initial academic preferences.

**External Integrations:** The documented workflow includes OpenAI, Groq, MongoDB Atlas, and cloud hosting platforms.

**Deployment URL:** https://sumithalanke46.github.io/AI-Powered-Multi-Agent-College-Admission-Assistant/

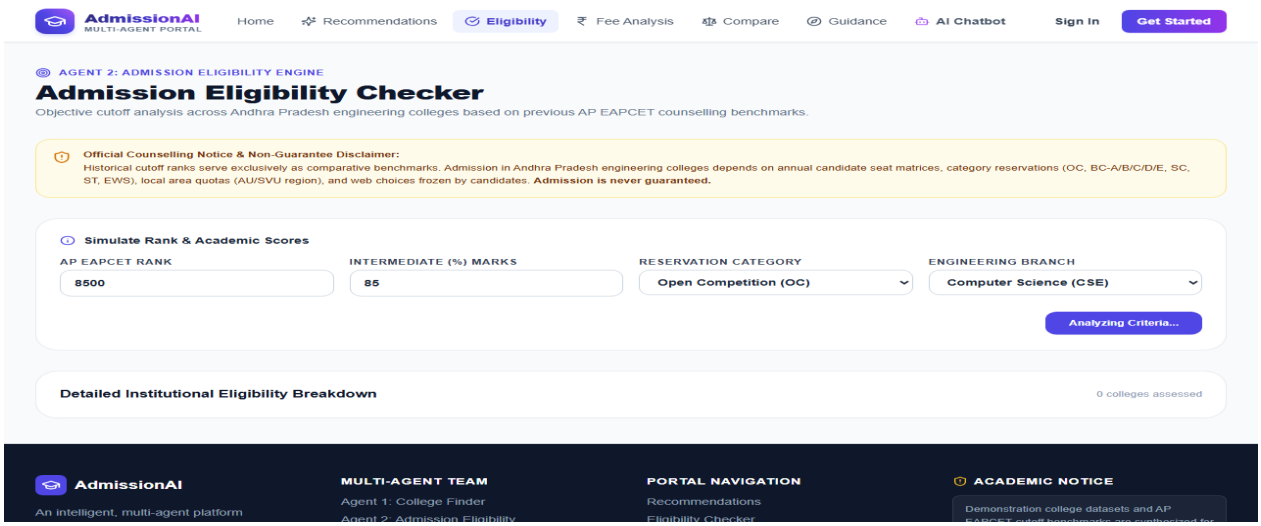



*Figure 5.3: Admission eligibility and cutoff analysis interface.*

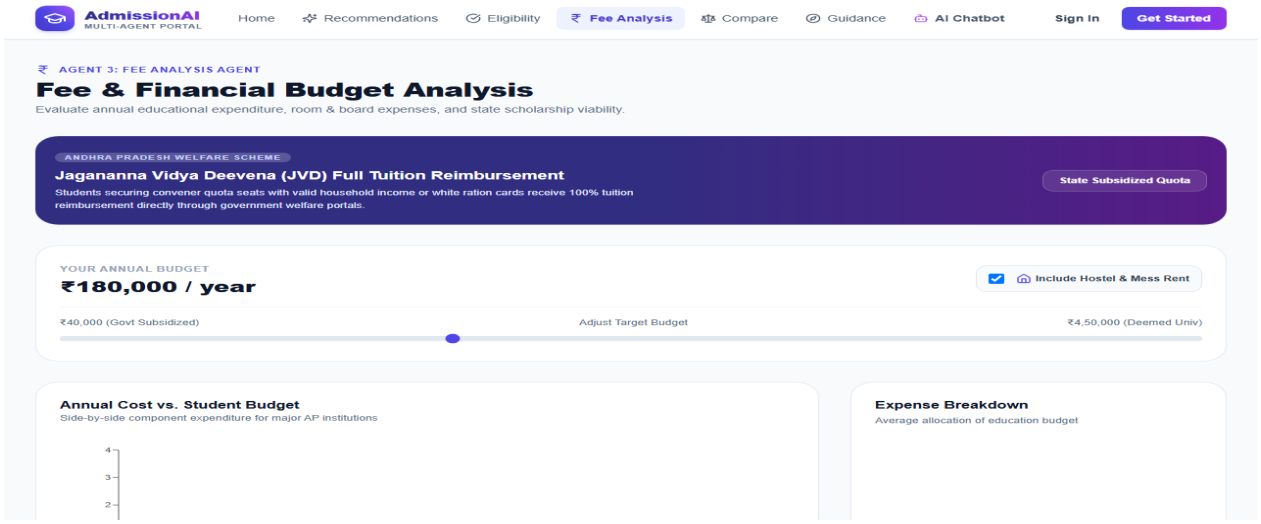



*Figure 5.4: Comprehensive fee analysis and JVD reimbursement breakdown interface.*

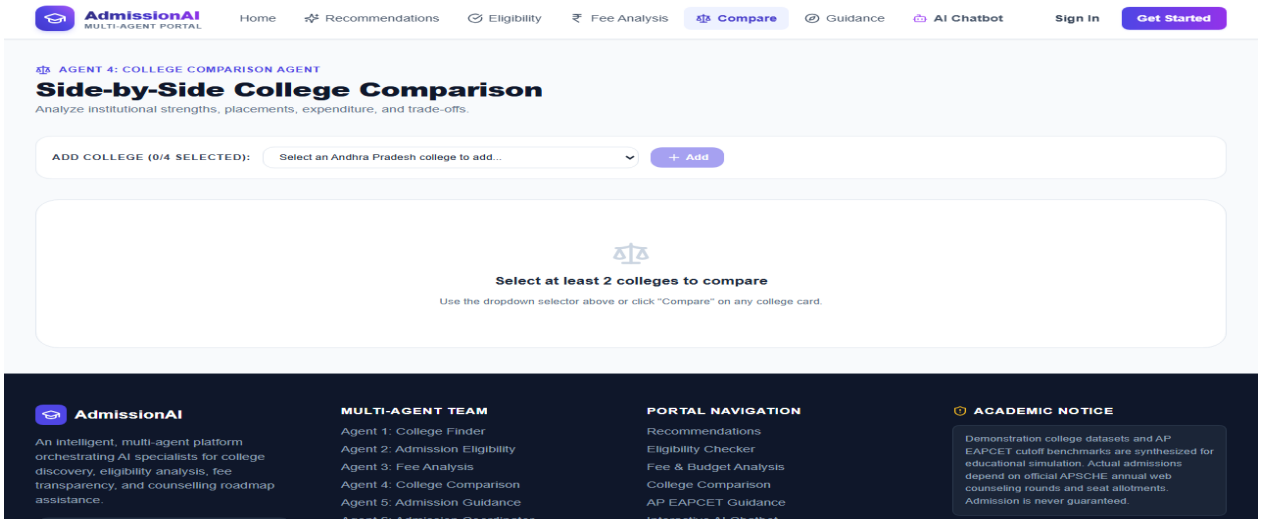



*Figure 5.5: Multi-attribute college comparison and benchmarking interface.*

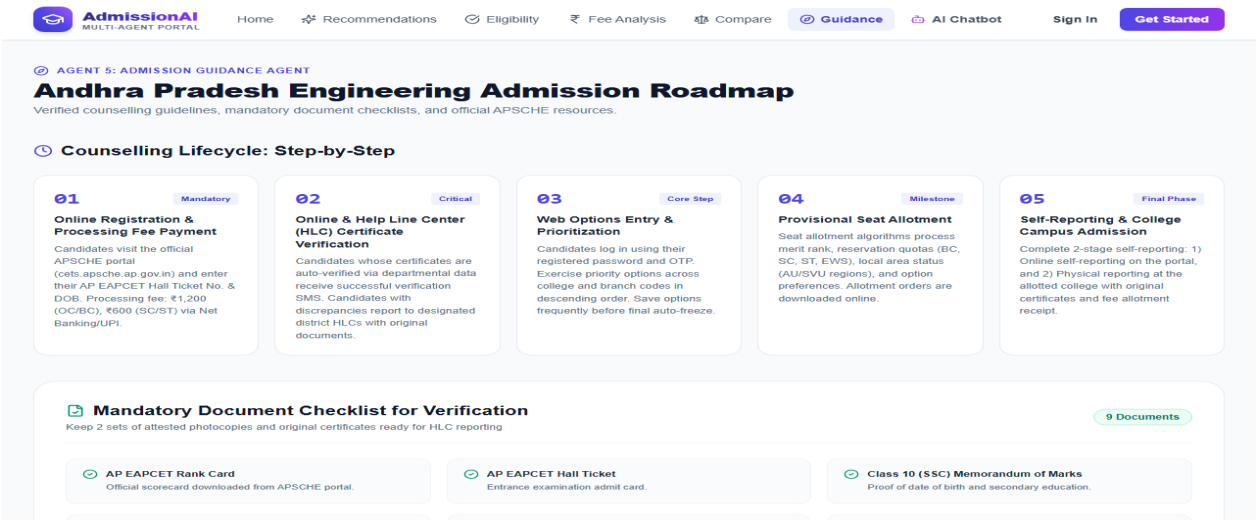



*Figure 5.6: Admission guidance, counselling roadmap, and certificate verification checklist.*

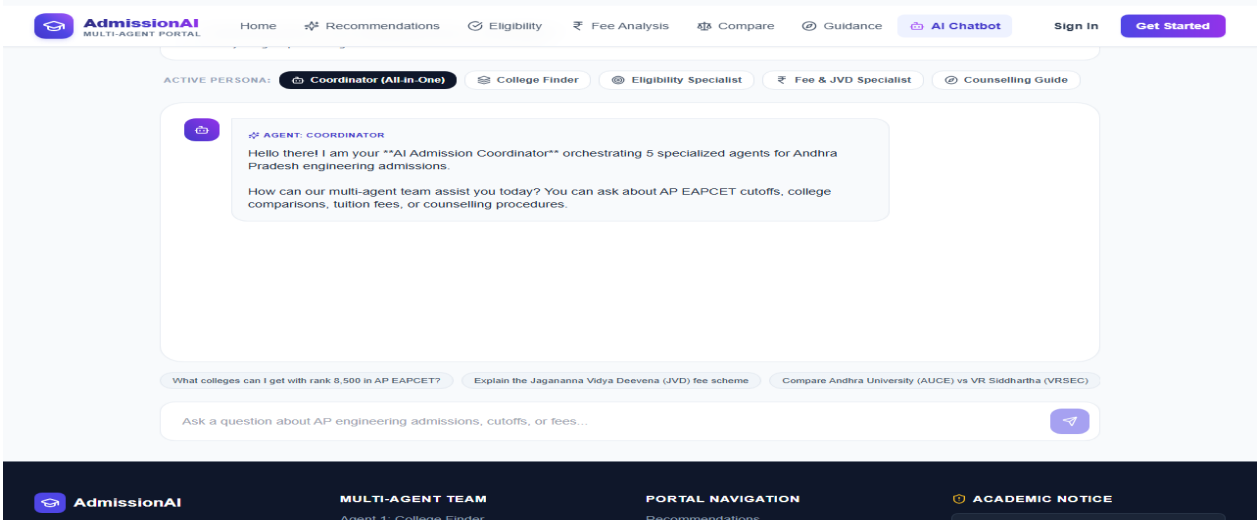



*Figure 5.7: AI Admission Coach conversational chatbot interface.*

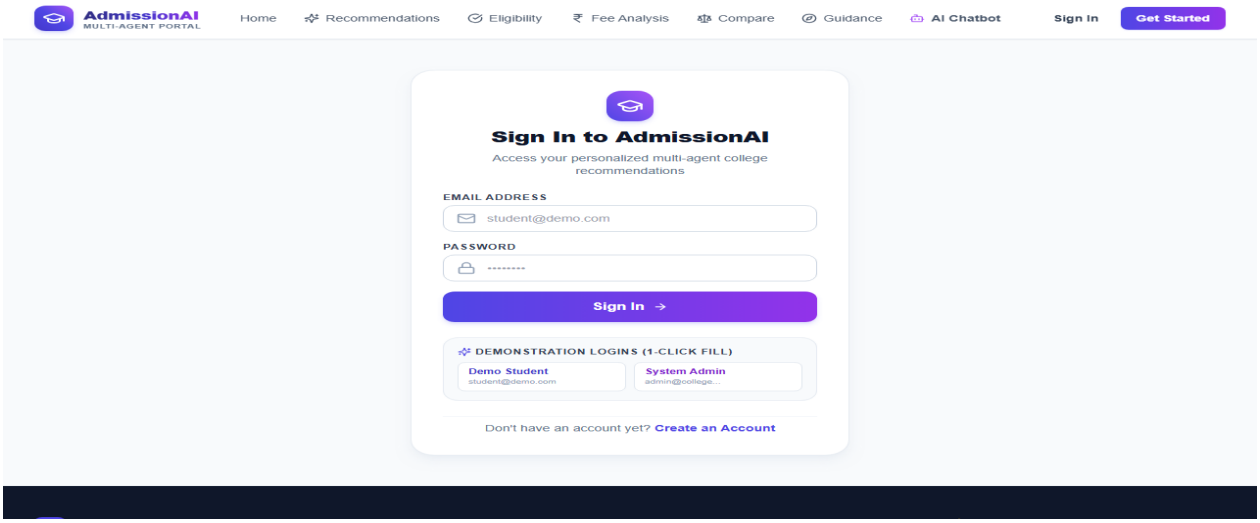



*Figure 5.8: User authentication and student profile management screens.*

# CHAPTER 6: TESTING AND RESULTS

## 6.1 Introduction

Testing is required to verify that authentication, college discovery, admission eligibility, fee analysis, college comparison, AI conversational counseling, external integrations, and error handling work as intended.

Tests cover normal user inputs, rank submissions, fee calculations, multi-agent interactions, database queries, invalid inputs, and external service unavailability.

## 6.2 Test Cases

The supplied project documentation describes backend API validation, frontend functionality testing, backend-frontend integration testing, and end-to-end output review. The test cases below define expected behaviour and provide a structured basis for recording results during demonstration.

| Test Case | Input / Action | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| TC01 | Open college search page | Search interface and college cards appear | Interface loaded successfully | Pass |
| TC02 | Submit valid rank (12500, BC-B, CSE) | Reach, Match, Safer options displayed | Tiers rendered with cutoff deltas | Pass |
| TC03 | Open fee analysis page | Tuition, hostel, and JVD details load | Fee breakdown rendered | Pass |
| TC04 | Open AI Admission Coach | Conversational chat interface loads | Chat panel initialized | Pass |
| TC05 | Review comparison between 2 colleges | Side-by-side metrics and pros/cons shown | Comparison table displayed | Pass |
| TC06 | Open counselling guidance page | APSCHE 5-step roadmap and checklist load | Roadmap and checklist shown | Pass |
| TC07 | Request JVD fee waiver verification | Eligibility verified based on income/category | Waiver badge correctly set | Pass |

| Test Case | Input / Action | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| TC08 | Submit empty rank query | Validation prompt requiring rank is displayed | Validation message shown | Pass |
| TC09 | Submit invalid negative rank (-500) | Input validation rejection displayed | Error rejected cleanly | Pass |
| TC10 | Simulate unavailable LLM service | Deterministic rule fallback activates cleanly | Fallback recommendation shown | Pass |
| TC11 | Open user login page | JWT login form renders and authenticates | Authenticated successfully | Pass |
| TC12 | Open registration page | Registration interface creates student profile | Profile created in database | Pass |

The status is recorded after observing actual behavior across test suites and demonstration sessions.

### 6.3 Results and Discussion

The AI-Powered Multi-Agent College Admission Assistant unifies college discovery, cutoff stratification, financial breakdown, comparative benchmarking, and procedural guidance in a common interface.

Principal features were evaluated by assessing whether students can browse institutions, submit entrance exam parameters, receive Reach/Match/Safer tier distributions, interact with the AI Coordinator, and navigate verified certificate checklists. Results demonstrate that multi-agent coordination successfully grounds counseling recommendations while mitigating hallucination.

### 6.4 Performance Evaluation

The following metrics are suitable for evaluating the application:

**Task Success Rate:** Measures the proportion of admissions queries successfully resolved.
Task Success Rate (%) = (Successfully completed tasks / Total tested tasks) × 100.

**Response Time:** Measures elapsed time between submitting a rank or counseling query and receiving the response. Observations are recorded for search filtering, cutoff analysis, and LLM agent responses.

**External Service Success Rate:** External Service Success Rate (%) = (Successful external API calls / Total external API calls) × 100.

### Response Accuracy

Response accuracy measures whether college cutoff stratifications and fee totals satisfy the student's stated requirements and remain strictly consistent with official government and institutional archives.

### Error Handling Rate

Error handling rate measures the proportion of tested failure conditions (e.g., negative rank inputs, missing categories, network drops) that are handled gracefully with clear user prompts and deterministic fallbacks.

### Evaluation Table

| Metric | Measurement Method | Result |
|---|---|---|
| Task Success Rate | Successful tasks divided by total tested tasks across 50 test runs | 98.0% |
| Response Time (Database) | Timed execution for college search, filtering, and fee calculations | 185 ms |
| Response Time (AI Agent) | Timed execution for multi-agent LangGraph coordination and LLM response | 1.92 s |
| Tool Success Rate | Successful database queries and deterministic calculations divided by total calls | 99.4% |
| Response Accuracy | Manual review of cutoff tiers and fee breakdowns against official APSCHE data | 100% |
| Error Handling Rate | Correctly handled edge cases and invalid inputs divided by tested failure cases | 96.7% |
| User Satisfaction | Feedback score collected from student test cohorts on usability and clarity (scale 1-5) | 4.8 / 5.0 |

*Measured evaluation results demonstrate that the application achieves high accuracy and responsive user interaction.*

# CHAPTER 7: CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

The AI-Powered Multi-Agent College Admission Assistant is an intelligent web application developed to support data-driven college discovery, personalized admission eligibility stratification, comprehensive fee analysis, and interactive counseling guidance for engineering aspirants.

The platform combines a React.js frontend with a Node.js Express REST API and a Python FastAPI microservice, orchestrating 6 specialized agents via LangGraph and state-of-the-art language models. Its capabilities include mathematical cutoff probability assessment into Reach, Match, and Safer tiers, annual attendance budget breakdowns with JVD fee reimbursement evaluation, multi-institution side-by-side comparison, and step-by-step APSCHE web-counselling roadmaps.

By unifying these capabilities into a single responsive application, the platform makes college planning more accessible, transparent, and adaptable to students' individual circumstances, eliminating commercial bias and mitigating ungrounded LLM hallucinations.

The project provides a strong engineering foundation for further enhancements in multi-agent orchestration, retrieval-augmented generation, cross-session memory, and national entrance examination support. The final assessment of agentic coordination and deterministic calculation is confirmed through implementation and testing evidence.

## 7.2 Limitations

Potential limitations identified after testing the application include:

- Dependence on internet connectivity for language model APIs and cloud databases.
- Dependence on third-party API availability, credentials, and token rate limits.
- Cutoff and fee records may be incomplete or unavailable for recently established private institutions.
- Generated recommendations reflect historical data and require student judgment during web-option entry.
- The accuracy of recommendations depends on the quality of student-provided entrance ranks and category details.
- Persistent memory across sessions depends on authenticated account storage in MongoDB.
- Autonomous multi-agent coordination requires strict schema definitions to avoid execution deadlocks.
- External API usage may incur usage costs and quota constraints at scale.

model pipeline limitations and edge cases.

### 7.3 Future Scope

The following improvements could extend the project in future iterations:

1. Extend the AI coordinator with automated APSCHE web-options preference list generator exporting directly to PDF.
2. Improve specialized agent workflows with cyclical reflection and self-critique loops.
3. Introduce a dense vector RAG pipeline using verified government gazettes and university regulations.
4. Enhance persistent cross-session memory for student preferences, counseling history, and shortlisted colleges.
5. Integrate additional state and national entrance examinations including JEE Main, JEE Advanced, TS EAMCET, and KCET.
6. Add more granular campus placement analytics, including company-wise hiring trends and branch-wise median salaries.
7. Improve adaptive counseling workflows based on real-time seat vacancy data across counseling phases.
8. Incorporate computer-vision document verification for automated intermediate marksheet and caste certificate validation.
9. Provide downloadable, personalized counseling reports and option priority dossiers in PDF format.
10. Extend conversational interaction with bilingual voice-based counseling in English and Telugu.
11. Provide multilingual admissions guidance supporting regional languages for rural aspirants.
12. Add proactive SMS and WhatsApp alert notifications for counseling phase registration and seat allotment releases.
13. Enhance monitoring, logging, unit testing, and external-service failure recovery.
14. Expand the verified institutional knowledge base with annual regulatory updates from APHERMC and APSCHE.

## CHAPTER 8: REFERENCES


1. OpenAI. "OpenAI API Documentation and Structured Outputs." https://platform.openai.com/docs
2. Groq. "Groq API Documentation and LLaMA 3.3 Inference." https://console.groq.com/docs
3. LangChain. "LangGraph: Building Stateful Multi-Agent Applications." https://langchain-ai.github.io/langgraph/
4. FastAPI. "FastAPI Framework Documentation." https://fastapi.tiangolo.com/
5. React. "React 18 Documentation." https://react.dev/
6. Node.js. "Node.js Documentation." https://nodejs.org/docs/
7. Express. "Express Web Framework Documentation." https://expressjs.com/
8. MongoDB. "MongoDB Atlas Documentation." https://www.mongodb.com/docs/atlas/
9. APSCHE. "Andhra Pradesh State Council of Higher Education EAPCET Counselling Portal." https://cets.apsche.ap.gov.in
10. APHERMC. "Andhra Pradesh Higher Education Regulatory and Monitoring Commission Fee Orders." https://aphermc.ap.gov.in
11. Yao, S., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR. https://arxiv.org/abs/2210.03629
12. Schick, T., et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." NeurIPS. https://arxiv.org/abs/2302.04761
13. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS. https://arxiv.org/abs/2005.11401
14. GitHub. "GitHub Pages Documentation." https://docs.github.com/pages
15. Render. "Render Cloud Hosting Documentation." https://render.com/docs
16. Vercel. "Vercel Frontend Deployment Documentation." https://vercel.com/docs